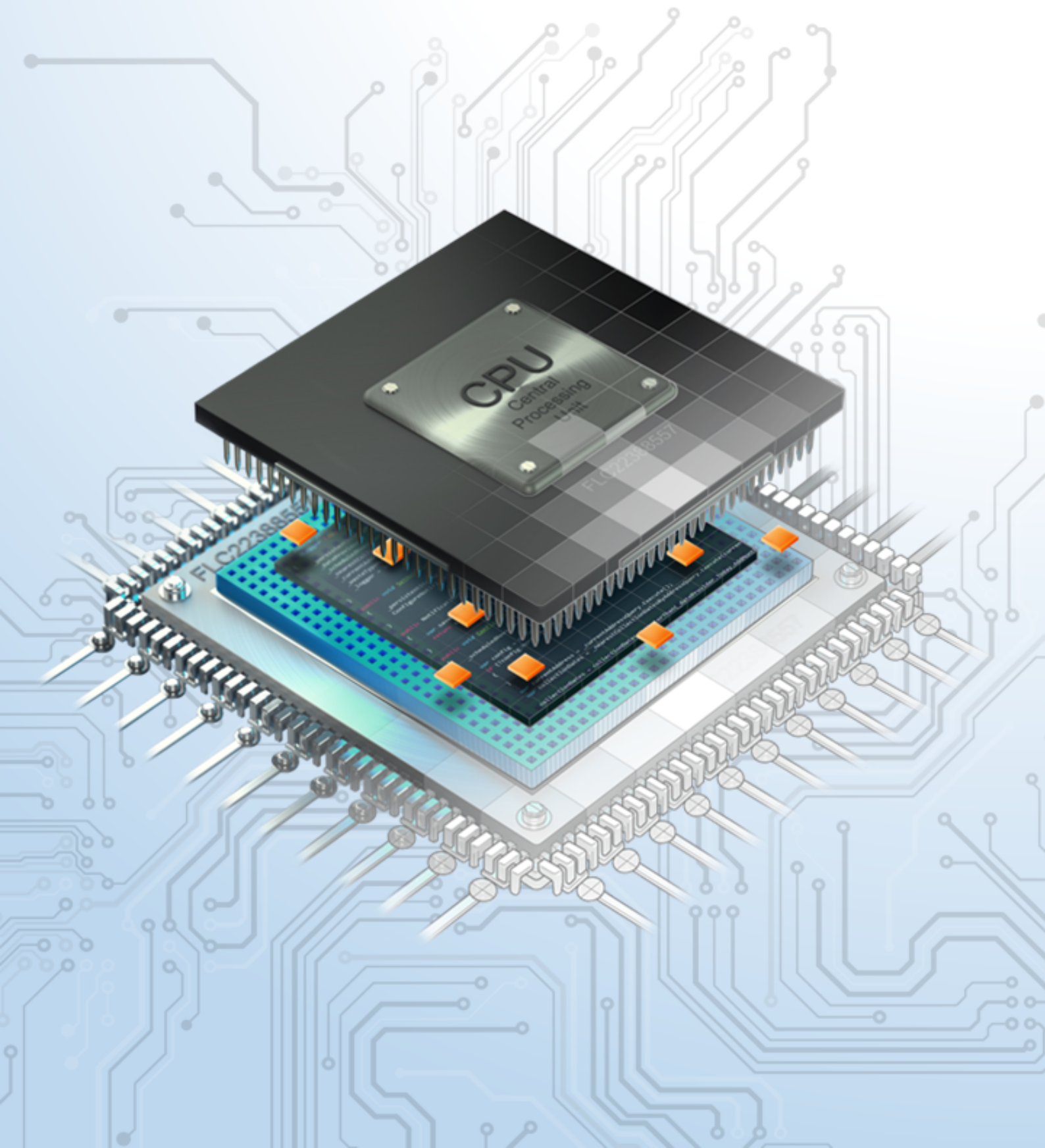




HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller





ĐẠI HỌC BÁCH KHOA - ĐHQG TP.HCM

KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH

BÁO CÁO THÍ NGHIỆM

VI ĐIỀU KHIỂN

LAB 2

Timer Interrupt and LED scanning

Giảng viên hướng dẫn	TS. Lê Trọng Nhân
Sinh viên thực hiện	Lê Anh Huy
Mã số sinh viên	2411184
Ngành	Kỹ thuật Máy tính
Trường	ĐH Bách Khoa - ĐHQG TPHCM

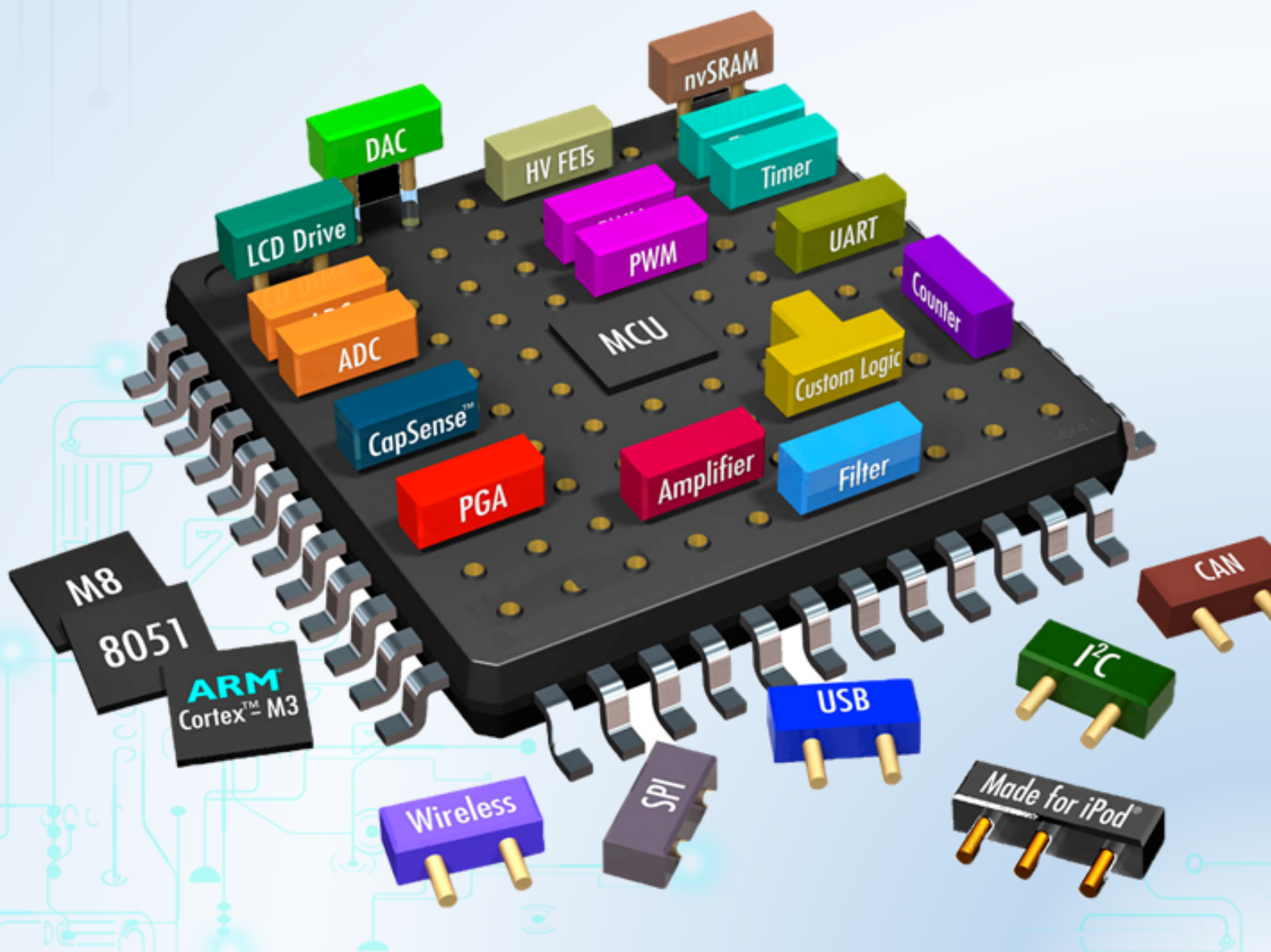
TP. HỒ CHÍ MINH, 2026

Mục lục

Chapter 1. Timer Interrupt and LED Scanning	4
1 REPORT	5
1.1 Exercise 1	5
1.2 Exercise 2	7
1.3 Exercise 3	8
1.4 Exercise 4	11
1.5 Exercise 5	12
1.6 Exercise 6	14
1.7 Exercise 7	17
1.8 Exercise 8	18
1.9 Exercise 9	21
1.10 Exercise 10	23

CHƯƠNG 1

Timer Interrupt and LED Scanning



1 REPORT

Timer 1 Configuration Analysis for Exercise 1 & Exercise 2

In the provided source code, the initialization function `MX_TIM1_Init()` configures Timer 1 with the following parameters:

- System Clock (f_{CLK}): 8 MHz (Internal High-Speed RC oscillator - HSI)
- Prescaler (PSC): 7999
- Counter Period / Auto-Reload Register (ARR): 499
- Counter Mode: Up-counting

The timer interrupt frequency (f_{INT}) and interrupt period (T_{INT}) are calculated using the standard STM32 Timer formulas:

$$f_{TIMER_CLK} = \frac{f_{CLK}}{\text{Prescaler} + 1} = \frac{8,000,000 \text{ Hz}}{7999 + 1} = \frac{8,000,000}{8000} = 1000 \text{ Hz} \quad (1 \text{ ms per tick}) \quad (1.1)$$

$$T_{INT} = \frac{(\text{Period} + 1)}{f_{TIMER_CLK}} = \frac{499 + 1}{1000 \text{ Hz}} = \frac{500}{1000} = 0.5 \text{ s} = 500 \text{ ms} \quad (1.2)$$

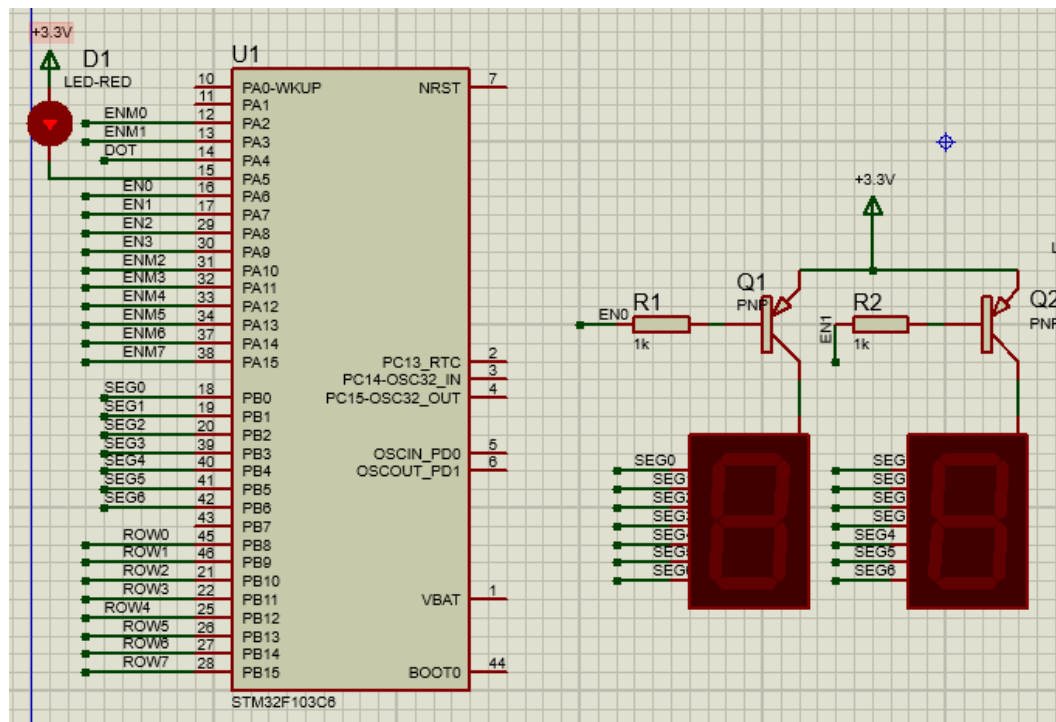
$$f_{INT} = \frac{1}{T_{INT}} = \frac{1}{0.5 \text{ s}} = 2 \text{ Hz} \quad (1.3)$$

1.1 Exercise 1

The first exercise shows how to interface multiple seven-segment LEDs to the STM32F103C6 microcontroller. Two 7-segment LEDs (Common Anode) are used to display numbers "1" and "2" sequentially with a switching time of 0.5s (500ms). **Report 1:** Capture your schematic from Proteus and show in the report.

Report 2: Present your source code in the `HAL_TIM_PeriodElapsedCallback` function.

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     if (htim->Instance == htim1.Instance) {
4         if (flag == 0) {
5             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
6             GPIO_PIN_SET); // Disable LED 2
7             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
8             GPIO_PIN_RESET); // Enable LED 1
9             LED7_Display(1);
10            flag = 1;
```



Hình 1.1: Simulation schematic in Proteus for Exercise 1

```

8         } else {
9             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
10                GPIO_PIN_SET); // Disable LED 1
11             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
12                GPIO_PIN_RESET); // Enable LED 2
13             LED7_Display(2);
14             flag = 0;
15         }
16     }
17 }

```

Program 1.1: Source code for Exercise 1 in timer interrupt callback

Short question: What is the frequency of the scanning process?

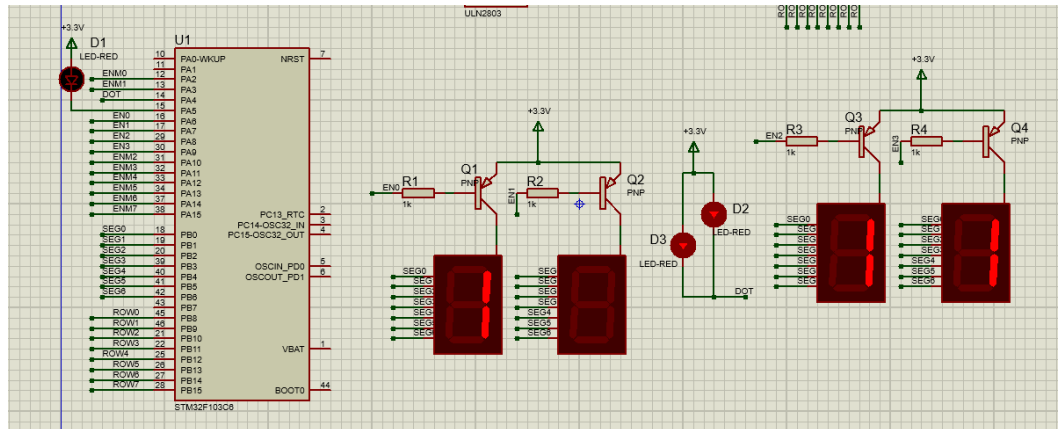
- **Answer:** Since each LED is turned on for 500 ms (0.5 s), one complete scanning cycle across both LEDs takes $T = 500 \text{ ms} + 500 \text{ ms} = 1000 \text{ ms} = 1 \text{ s}$.
- Therefore, the scanning frequency is:

$$f = \frac{1}{T} = \frac{1}{1 \text{ s}} = 1 \text{ Hz}$$

1.2 Exercise 2

Extend the system to 4 seven-segment LEDs and two LEDs (connected to PA4, labeled as **DOT**) in the middle. The DOT LEDs blink every second, while numbers 1, 2, 3, and 0 are displayed across the 4 seven-segment LEDs sequentially with a switching time of 500ms for each segment.

Report 1: Capture your schematic from Proteus and show in the report.



Hình 1.2: Simulation schematic in Proteus for Exercise 2

Report 2: Present your source code in the `HAL_TIM_PeriodElapsedCallback` function.

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     if (htim->Instance == htim1.Instance) {
4         // Toggle PA4 (DOT LEDs) on every interrupt trigger
5         // (every 500ms / 1s period cycle)
6         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
7
8         if (flag == 0b00) {
9             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7 |
10             GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);
11             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
12             GPIO_PIN_RESET); // Enable 1st LED
13             LED7_Display(1);
14             flag = 0b01;
15         } else if (flag == 0b01) {
16             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
17             GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);
18             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
19             GPIO_PIN_RESET); // Enable 2nd LED
20             LED7_Display(2);
21             flag = 0b10;
22         } else if (flag == 0b10) {
23             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
24             GPIO_PIN_7 | GPIO_PIN_9, GPIO_PIN_SET);
25             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
26             GPIO_PIN_RESET); // Enable 3rd LED
27             LED7_Display(3);
28             flag = 0b11;
29         } else if (flag == 0b11) {
30             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
31             GPIO_PIN_7 | GPIO_PIN_8, GPIO_PIN_SET);
32             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9,
33             GPIO_PIN_RESET); // Enable 4th LED
34             LED7_Display(4);
35             flag = 0b00;
36         }
37     }
38 }
```

```

18         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8 ,
GPIO_PIN_RESET); // Enable 3rd LED
19         LED7_Display(3);
20         flag = 0b11;
21     } else {
22         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
GPIO_PIN_7 | GPIO_PIN_8, GPIO_PIN_SET);
23         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9 ,
GPIO_PIN_RESET); // Enable 4th LED
24         LED7_Display(0);
25         flag = 0b00;
26     }
27 }
28 }

```

Program 1.2: Source code for Exercise 2 in timer interrupt callback

Short question: What is the frequency of the scanning process?

- **Answer:** Since each LED is enabled sequentially for 500 ms, one full refresh cycle for all 4 seven-segment LEDs takes $T = 4 \times 500 \text{ ms} = 2000 \text{ ms} = 2 \text{ s}$.
- Therefore, the scanning frequency is:

$$f = \frac{1}{T} = \frac{1}{2 \text{ s}} = 0.5 \text{ Hz}$$

1.3 Exercise 3

Exercise 3 modularizes the 7-segment display logic by implementing the function `update7SEG(int index)`. This function takes an index parameter and displays the corresponding value from an array `led_buffer[4] = {1, 2, 3, 4}` onto the designated 7-segment LED display.

Timer 1 Configuration Analysis for Exercise 3

In Exercise 3, Timer 1 retains the same configuration as Exercise 1 and 2:

- System Clock (f_{CLK}): 8 MHz
- Prescaler (PSC): 7999
- Counter Period (ARR): 499

The interrupt period (T_{INT}) and frequency (f_{INT}) are calculated as:

$$f_{TIMER_CLK} = \frac{8,000,000 \text{ Hz}}{7999 + 1} = 1000 \text{ Hz} \quad (1 \text{ ms per tick}) \quad (1.4)$$

$$T_{INT} = \frac{499 + 1}{1000 \text{ Hz}} = \frac{500}{1000} = 0.5 \text{ s} = 500 \text{ ms} \quad (1.5)$$

Report 1: Present the source code of the `update7SEG` function.

```

1 void update7SEG(int idx) {
2     switch (idx) {
3         case 0:
4             LED7_Display(led_buffer[0]);
5             break;
6         case 1:
7             LED7_Display(led_buffer[1]);
8             break;
9         case 2:
10            LED7_Display(led_buffer[2]);
11            break;
12        case 3:
13            LED7_Display(led_buffer[3]);
14            break;
15        default:
16            break;
17    }
18 }

```

Program 1.3: Source code for update7SEG function

Report 2: Present the source code in the **HAL_TIM_PeriodElapsedCallback**.

```

1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     if (htim->Instance == htim1.Instance) {
4         if (id >= MAX_LED) id = 0;
5         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4); // Toggle
6         DOT LED
7
8         if (flag == 0b00) {
9             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7 |
10             GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);
11             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
12             GPIO_PIN_RESET);
13             update7SEG(id++);
14             flag = 0b01;
15         } else if (flag == 0b01) {
16             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
17             GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);
18             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
19             GPIO_PIN_RESET);
20             update7SEG(id++);
21             flag = 0b10;
22         } else if (flag == 0b10) {
23             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
24             GPIO_PIN_7 | GPIO_PIN_9, GPIO_PIN_SET);
25             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
26             GPIO_PIN_RESET);
27             update7SEG(id++);
28             flag = 0b11;
29         } else if (flag == 0b11) {
30             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
31             GPIO_PIN_7 | GPIO_PIN_8, GPIO_PIN_SET);
32             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9,
33             GPIO_PIN_RESET);
34             update7SEG(id++);
35             flag = 0b00;
36         }
37     }
38 }

```

```

21         } else {
22             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 |
GPIO_PIN_7 | GPIO_PIN_8, GPIO_PIN_SET);
23             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9,
GPIO_PIN_RESET);
24             update7SEG(id++);
25             flag = 0b00;
26         }
27     }
28 }

```

Program 1.4: Source code for Exercise 3 in timer interrupt callback

Scanning Frequency Calculation for Exercise 3:

- Switching period per LED: $T_{switch} = 500 \text{ ms} = 0.5 \text{ s}$.
- Total cycle time for 4 LEDs: $T_{scan} = 4 \times 500 \text{ ms} = 2000 \text{ ms} = 2.0 \text{ s}$.
- Scanning frequency: $f_{scan} = \frac{1}{2.0 \text{ s}} = 0.5 \text{ Hz}$.

1.4 Exercise 4

Exercise 4 modifies the timer period to increase the scanning frequency of the 4 seven-segment LEDs to 1 Hz while keeping the DOT blinking interval at 1 s (0.5 s ON, 0.5 s OFF).

Timer 1 Configuration Analysis for Exercise 4

In `MX_TIM1_Init()`, the period register `ARR` is changed to 249:

- System Clock (f_{CLK}): 8 MHz
- Prescaler (PSC): 7999
- Counter Period (ARR): 249

The updated timer interrupt period (T_{INT}) is:

$$T_{INT} = \frac{\text{Period} + 1}{f_{TIMER_CLK}} = \frac{249 + 1}{1000 \text{ Hz}} = \frac{250}{1000} = 0.25 \text{ s} = 250 \text{ ms} \quad (1.6)$$

Scanning Frequency Analysis (1 Hz Target):

- Each interrupt occurs every 250 ms, switching to the next LED.
- Full scanning cycle duration for 4 LEDs: $T_{scan} = 4 \times 250 \text{ ms} = 1000 \text{ ms} = 1.0 \text{ s}$.
- Overall display scanning frequency: $f_{scan} = \frac{1}{1.0 \text{ s}} = 1 \text{ Hz}$.
- To maintain the DOT LED (PA4) toggling every 500 ms, a software counter (`timer_counter`) toggles PA4 every 2 interrupt cycles ($2 \times 250 \text{ ms} = 500 \text{ ms}$).

Report 1: Present the source code in the `HAL_TIM_PeriodElapsedCallback`.

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     if (htim->Instance == htim1.Instance) {
4         // Disable all 7-segment enable pins (PA6, PA7, PA8, PA9)
5         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 | GPIO_PIN_7 |
6         GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);
7
8         // Turn on the current LED and update its display
9         switch (id) {
10             case 0:
11                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
12                 GPIO_PIN_RESET);
13                 update7SEG(0);
14                 break;
15             case 1:
```

```

13         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
GPIO_PIN_RESET);
14         update7SEG(1);
15         break;
16     case 2:
17         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
GPIO_PIN_RESET);
18         update7SEG(2);
19         break;
20     case 3:
21         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9,
GPIO_PIN_RESET);
22         update7SEG(3);
23         break;
24     }
25
26     // Advance to next LED digit index
27     id = (id + 1) % MAX_LED;
28
29     // Toggle DOT LED (PA4) every 500ms (every 2 timer
interrupts of 250ms)
30     timer_counter++;
31     if (timer_counter >= 2) {
32         timer_counter = 0;
33         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
34     }
35 }
36 }

```

Program 1.5: Source code for Exercise 4 in timer interrupt callback

1.5 Exercise 5

Exercise 5 implements a real-time digital clock display showing hours and minutes (*HH:MM*) using four 7-segment LED displays. The clock increments the second, minute, and hour values dynamically in real time and updates the global buffer `led_buffer[4]` via the helper function `updateClockBuffer(hour, minute)`.

Timer 1 Configuration Analysis for Exercise 5

In Exercise 5, Timer 1 is configured in `MX_TIM1_Init()` with the following parameters:

- System Clock (f_{CLK}): 8 MHz
- Prescaler (PSC): 7999
- Counter Period (ARR): 249

The timer interrupt period (T_{INT}) and frequency (f_{INT}) are calculated as:

$$f_{TIMER_CLK} = \frac{8,000,000 \text{ Hz}}{7999 + 1} = 1000 \text{ Hz} \quad (1 \text{ ms per clock tick}) \quad (1.7)$$

$$T_{INT} = \frac{249 + 1}{1000 \text{ Hz}} = \frac{250}{1000} = 0.25 \text{ s} = 250 \text{ ms} \quad (1.8)$$

Scanning and Clock Update Analysis:

- **Display Scanning:** The interrupt triggers every 250 ms to switch to the next 7-segment display. A full scanning cycle across all 4 LEDs takes $4 \times 250 \text{ ms} = 1000 \text{ ms} = 1.0 \text{ s}$, corresponding to a display scanning frequency of $f_{scan} = 1.0 \text{ Hz}$.
- **Clock Logic Execution:** The timekeeping counter increments every second in the main loop or interrupt context. The `updateClockBuffer(hour, minute)` function decomposes the current time into individual digits:

```
led_buffer[0] = hour/10   (Tens digit of Hour)
led_buffer[1] = hour%10   (Units digit of Hour)
led_buffer[2] = minute/10 (Tens digit of Minute)
led_buffer[3] = minute%10 (Units digit of Minute)
```

Report 1: Present the source code executed inside the **main loop** (`while(1)`).

```
1  /* Inside main() function */
2  updateClockBuffer(hour, minute);
3
4  while (1) {
5      /* Exercise 5: Main Loop Execution Logic */
6      second++;
7      if (second >= 60) {
8          second = 0;
9          minute++;
10     }
11     if (minute >= 60) {
12         minute = 0;
13         hour++;
14     }
15     if (hour >= 24) {
16         hour = 0;
17     }
18
19     // Update the led_buffer with the new time
20     updateClockBuffer(hour, minute);
21
22     // Delay 1 second between clock increments
23     HAL_Delay(1000);
24 }
```

Program 1.6: Source code running inside main loop for Exercise 5

1.6 Exercise 6

The primary target of Exercise 6 is to replace blocking delay calls like `HAL_Delay()` with a non-blocking **Software Timer** architecture. Reducing code processing execution inside the hardware timer ISR minimizes interrupt latency and prevents nested interrupt crashes.

In this system configuration, Timer 1 is configured to generate an interrupt every **1 ms**. The software timer parameter `TIMER_CYCLE` is set to 10 ms.

Timer 1 Configuration Analysis for Exercise 6

Based on `MX_TIM1_Init()` in `main_6.c`:

- System Clock (f_{CLK}): 8 MHz
- Prescaler (PSC): 7999
- Counter Period (ARR): 9

The hardware timer interrupt period (T_{INT}) is calculated as:

$$f_{TIMER_CLK} = \frac{8,000,000 \text{ Hz}}{7999 + 1} = 1000 \text{ Hz} \quad (1 \text{ tick per ms}) \quad (1.9)$$

$$T_{INT} = \frac{9 + 1}{1000 \text{ Hz}} = \frac{10}{1000} = 0.001 \text{ s} = 1 \text{ ms} \quad (1.10)$$

Software Timer Operation Mechanism

When calling `setTimer0(1000)`, the software counter is calculated as:

$$\text{timer0_counter} = \frac{\text{duration}}{\text{TIMER_CYCLE}} = \frac{1000 \text{ ms}}{10 \text{ ms}} = 100 \quad (1.11)$$

Since the hardware timer fires every 1 ms, it requires 10 hardware interrupt cycles ($10 \times 1 \text{ ms} = 10 \text{ ms}$) to decrement `timer0_counter` by 1. Thus, to count down from 100 to 0, it takes $100 \times 10 \text{ ms} = 1000 \text{ ms}$ (1 second). Once `timer0_counter` reaches 0, `timer0_flag` is set to 1, signaling the main loop to perform the task.

Source Code Implementation

1. Software Timer Definition & Helper Functions:

```
1 int timer0_counter = 0;
2 int timer0_flag = 0;
3 int TIMER_CYCLE = 10;
4
5 void setTimer0(int duration) {
6     timer0_counter = duration / TIMER_CYCLE;
```

```

7     timer0_flag = 0;
8 }
9
10 void timer_run() {
11     if (timer0_counter > 0) {
12         timer0_counter--;
13         if (timer0_counter == 0) timer0_flag = 1;
14     }
15 }

```

Program 1.7: Software timer variables and functions in main_6.c

2. Timer Interrupt Callback Handler:

```

1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     if (htim->Instance == htim1.Instance) {
4         timer_run();
5     }
6 }

```

Program 1.8: Timer interrupt handler invoking timer_run()

3. Main Loop Logic (while(1)):

```

1 /* Inside main() before while(1) */
2 setTimer0(1000);
3
4 /* Inside while(1) infinite loop */
5 while (1) {
6     if (timer0_flag == 1) {
7         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
8         setTimer0(2000);
9     }
10 }

```

Program 1.9: Non-blocking LED toggle logic inside main loop

Answers to Report Questions

Report 1: If Line 1 (setTimer0(1000);) before the while(1) loop is missing, what happens after that and why?

- **Behavior:** The LED on PA5 will **never toggle** and will remain in its initial state indefinitely.
- **Explanation:** Initially, global variables are initialized as timer0_counter = 0 and timer0_flag = 0. If setTimer0(1000) is not called before entering the while(1) loop:

1. `timer0_counter` remains 0, so `timer_run()` inside the interrupt will never decrement it or set `timer0_flag = 1`.
2. Inside `while(1)`, the condition `if (timer0_flag == 1)` will evaluate to **false** forever.
3. As a result, the code inside the `if` block is never executed, and PA5 never toggles.

Report 2: If Line 1 of the code above is changed to `setTimer0(1);`, what happens after that and why?

- **Behavior:** The LED on PA5 never toggles.
- **Explanation:**

1. Executing `setTimer0(1)` performs integer division:

$$\text{timer0_counter} = \frac{1}{10} = 0 \quad (1.12)$$

2. Because `timer0_counter` is set directly to 0, when `timer_run()` runs or when evaluated, `timer0_flag` is never triggered or never set to 1 forever due to value 0 initially preventing the if-condition following `timer0_counter > 0` to reach the establishment of flag to 1 for toggling.
2. The main loop instantly does not detect `timer0_flag == 1` to toggle PA5, and forbid then executing `setTimer0(2000)`, which will properly set `timer0_counter = 2000/10 = 200` (2000 ms) after. Afterwards, the LED usually remains the same initial state and not toggles.

Report 3: If Line 1 of the code above is changed to `setTimer0(10);`, what is changed compared to the first two questions and why?

- **Behavior:** The LED performs toggled immediately state compared to the question 1 with then experiencing 1 second, while the state in question 2 always remain the first state and never vary different state.
- **Explanation:**

1. Executing `setTimer0(10)` sets:

$$\text{timer0_counter} = \frac{10}{10} = 1 \quad (1.13)$$

2. Question 1 led will toggle when time exposure throughout 1000 ms after switching then 2000 ms. However, question 2 will frequently maintain the remaining state firstly forever.

1.7 Exercise 7

Upgrade the source code in Exercise 5 (update values for hour, minute and second) by using the software timer and remove the HAL_Delay function at the end. Moreover, the DOT (connected to PA4) of the digital clock is also moved to main function.

Report 1: Present the source code executed inside the **Timer Interrupt Callback Handler**.

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     if (htim->Instance == htim1.Instance) {
4         // Disable all 7-segment enable pins (PA6, PA7, PA8, PA9)
5         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 | GPIO_PIN_7 |
6         GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);
7
8         // Turn on the current LED and update its display
9         buffer
10         switch (id) {
11             case 0:
12                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
13                 GPIO_PIN_RESET);
14                 update7SEG(0);
15                 break;
16             case 1:
17                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
18                 GPIO_PIN_RESET);
19                 update7SEG(1);
20                 break;
21             case 2:
22                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
23                 GPIO_PIN_RESET);
24                 update7SEG(2);
25                 break;
26             case 3:
27                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9,
28                 GPIO_PIN_RESET);
29                 update7SEG(3);
30                 break;
31         }
32
33         // Advance to next LED digit index
34         id = (id + 1) % MAX_LED;
35
36         // Toggle DOT LED (PA4) every 1000ms (every 4 timer
37         interrupts of 250ms)
38         timer_counter++;
```

```

31     if (timer_counter >= 4) {
32         timer_counter = 0;
33         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
34         second++;
35         if (second >= 60) {
36             second = 0;
37             minute++;
38             if (minute >= 60) {
39                 minute = 0;
40                 hour++;
41             }
42             if(hour >= 24) hour = 0 ;
43         }
44         updateClockBuffer(hour, minute);
45     }
46 }
47 }
48 }
49 }

```

Program 1.10: Source code running in Timer Interrupt Callback for Exercise 5

1.8 Exercise 8

Exercise 8 combines real-time digital clock logic with multi-LED multiplexing using two independent software timers:

- **Software Timer 0 (timer0):** Controls the 1-second system clock tick, updates time variables (hour, minute, second), toggles the status LED, and updates the LED buffer.
- **Software Timer 1 (timer1):** Controls the high-frequency LED 7-segment multiplexing display sweep (display_7_LEDS()) to prevent visible flickering.

Timer Configuration Analysis

From the configuration in `main_7.c`:

- Clock Source: $f_{CLK} = 8 \text{ MHz}$
- Prescaler (PSC): 7999
- Counter Period (ARR): 9
- Software Timer Base Cycle (TIMER_CYCLE): 10 ms

The hardware interrupt interval (T_{INT}) is calculated as:

$$f_{TIMER} = \frac{8,000,000 \text{ Hz}}{7999 + 1} = 1000 \text{ Hz} \quad (1 \text{ interrupt every } 1 \text{ ms}) \quad (1.14)$$

$$T_{INT} = \frac{9+1}{1000 \text{ Hz}} = 1 \text{ ms} \quad (1.15)$$

Every 10 ms ($10 \times 1 \text{ ms}$ interrupts), the function `timer_run()` decrements active software timers:

$$\text{timer0_counter} = \frac{1000 \text{ ms}}{10 \text{ ms}} = 100 \quad (1 \text{ second delay}) \quad (1.16)$$

$$\text{timer1_counter} = \frac{20 \text{ ms}}{10 \text{ ms}} = 2 \quad (20 \text{ ms refresh delay}) \quad (1.17)$$

Source Code Implementation

1. Dual Software Timers Core Logic:

```

1 int timer0_counter = 0, timer0_flag = 0;
2 int timer1_counter = 0, timer1_flag = 0;
3 int TIMER_CYCLE = 10;
4
5 void setTimer0(int duration) {
6     timer0_counter = duration / TIMER_CYCLE;
7     timer0_flag = 0;
8 }
9
10 void setTimer1(int duration) {
11     timer1_counter = duration / TIMER_CYCLE;
12     timer1_flag = 0;
13 }
14
15 void timer_run() {
16     if (timer0_counter > 0) {
17         timer0_counter--;
18         if (timer0_counter == 0) timer0_flag = 1;
19     }
20     if (timer1_counter > 0) {
21         timer1_counter--;
22         if (timer1_counter == 0) timer1_flag = 1;
23     }
24 }
25
26 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
27 {
28     if (htim->Instance == htim1.Instance) {
29         timer_run();
30     }
31 }

```

Program 1.11: Software Timer definitions and execution in `main_7.c`

2. Clock Buffer Update and Multiplexing Functions:

```

1 void updateClockBuffer(int hour, int minute) {
2     led_buffer[0] = hour / 10;
3     led_buffer[1] = hour % 10;
4     led_buffer[2] = minute / 10;
5     led_buffer[3] = minute % 10;
6 }
7
8 void display_7_LEDS() {
9     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6 | GPIO_PIN_7 |
10     GPIO_PIN_8 | GPIO_PIN_9, GPIO_PIN_SET);
11     update7SEG(id);
12     switch (id) {
13         case 0: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
14         GPIO_PIN_RESET); break;
15         case 1: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
16         GPIO_PIN_RESET); break;
17         case 2: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
18         GPIO_PIN_RESET); break;
19         case 3: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9,
20         GPIO_PIN_RESET); break;
21     }
22     id = (id + 1) % MAX_LED;
23 }

```

Program 1.12: LED Multiplexing and Clock Buffer Management

3. Main Task Scheduling Loop (while(1)):

```

1 /* In main() initialization */
2 setTimer0(10);
3 setTimer1(500);
4
5 /* In while(1) loop */
6 while (1) {
7     // Task 1: 7-Segment Multiplexing Sweep (Every 20 ms)
8     if (timer1_flag == 1) {
9         timer1_flag = 0;
10        setTimer1(20);
11        display_7_LEDS();
12    }
13
14    // Task 2: Real-time Digital Clock Calculation (Every
15    1000 ms)
16    if (timer0_flag == 1) {
17        timer0_flag = 0;
18        setTimer0(1000);
19        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
20
21        second++;
22        if (second >= 60) {
23            second = 0;

```

```

23         minute++;
24     }
25     if (minute >= 60) {
26         minute = 0;
27         hour++;
28     }
29     if (hour >= 24) hour = 0;
30
31     updateClockBuffer(hour, minute);
32 }
33 }

```

Program 1.13: Non-blocking main loop execution for Exercise 8

1.9 Exercise 9

The objective of Exercise 9 is to implement the scanning method for an 8×8 LED Matrix display using a hardware-timer-based software timer mechanism.

1. Timer Hardware Configuration & Timing Calculations

The system hardware timer TIM1 is configured with the following parameters based on the system clock setup in `main.c`:

- **System Clock Frequency (f_{CK_PSC}):** 8 MHz (Internal High-Speed RC Oscillator - HSI)
- **Timer Prescaler (PSC):** 799 $\implies$ PSC + 1 = 800
- **Auto-Reload Register (ARR):** 9 $\implies$ ARR + 1 = 10

The interrupt frequency of TIM1 (f_{timer}) and its corresponding period (T_{timer}) are calculated as:

$$f_{timer} = \frac{f_{CK_PSC}}{(PSC + 1) \times (ARR + 1)} = \frac{8,000,000}{800 \times 10} = 1000 \text{ Hz} = 1 \text{ kHz} \quad (1.18)$$

$$T_{timer} = \frac{1}{f_{timer}} = \frac{1}{1000 \text{ Hz}} = 1 \text{ ms} \quad (1.19)$$

Thus, the base tick of the software timer architecture is `TIMER_CYCLE = 1 ms`. Every 1 ms, the hardware interrupt callback `HAL_TIM_PeriodElapsedCallback()` invokes `timer_run()`, which decrements the software counter variables.

2. Solution Description & Software Architecture

To scan the 8×8 LED Matrix:

- **Hardware Pin Mapping:**

- **Column Control (GPIOA):** Pins PA2, PA3, PA10, PA11, PA12, PA13, PA14, PA15 are configured as active-LOW column enable lines.
- **Row Control (GPIOB):** Pins PB8..PB15 drive the row data (active-LOW via BSRR bit mask).
- **Scanning Mechanism:** Setting `setTimer0(2)` configures Software Timer 0 to expire every 2 ms (2 ticks $\times$ 1 ms). When `timer0_flag` is set, the system calls `updateLEDMatrix(index_led_matrix)` to activate the next column and update row data.
- **Refresh Rate Calculation:** Scanning 8 columns at 2 ms per column results in a complete frame refresh period of:

$$T_{frame} = 8 \times 2 \text{ ms} = 16 \text{ ms} \Rightarrow f_{frame} = \frac{1}{16 \text{ ms}} = 62.5 \text{ Hz} \quad (1.20)$$

This refresh rate exceeds 50 Hz, ensuring visual persistence without flickering.

3. Source Code Implementation

Software Timer and Hardware Interrupt Setup:

```

1 int TIMER_CYCLE = 1; // 1 tick = 1ms
2
3 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
4 {
5     if (htim->Instance == htim1.Instance) {
6         timer_run(); // Executed every 1ms
7     }
8 }
9
10 void setTimer0(int duration) {
11     timer0_counter = duration / TIMER_CYCLE; // duration in
12     ms
13     timer0_flag = 0;
14 }
15
16 void timer_run(void) {
17     if (timer0_counter > 0) {
18         timer0_counter--;
19         if (timer0_counter == 0) timer0_flag = 1;
20     }
21     if (timer1_counter > 0) {
22         timer1_counter--;
23         if (timer1_counter == 0) timer1_flag = 1;
24     }
25 }

```

Program 1.14: TIM1 Interrupt Callback and Software Timer Subsystem

LED Matrix Display Function:

```

1  const int MAX_LED_MATRIX = 8;
2  int index_led_matrix = 0;
3
4  /* Bitmap character "A" representation */
5  uint8_t matrix_buffer[8] = { 0x00, 0x7C, 0x0A, 0x09, 0x09,
    0x0A, 0x7C, 0x00 };
6
7  static const uint16_t COL_PINS[8] = {
8      GPIO_PIN_2, GPIO_PIN_3, GPIO_PIN_10, GPIO_PIN_11,
9      GPIO_PIN_12, GPIO_PIN_13, GPIO_PIN_14, GPIO_PIN_15
10 };
11
12 #define COL_MASK (GPIO_PIN_2 | GPIO_PIN_3 | GPIO_PIN_10 |
    GPIO_PIN_11 | \
13
14                     GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
15                     GPIO_PIN_15)
16 #define ROW_MASK 0xFF00U    /* PB8..PB15 */
17
18 void updateLEDMatrix(int index) {
19     /* 1. Turn off all columns (SET = HIGH) to prevent
20     ghosting */
21     HAL_GPIO_WritePin(GPIOA, COL_MASK, GPIO_PIN_SET);
22
23     /* 2. Write inverted row pattern to PB8..PB15 via BSRR
24     */
25     uint16_t rows = ((uint16_t)((uint8_t)~matrix_buffer[
26     index])) << 8;
27     GPIOB->BSRR = (uint32_t)rows | ((uint32_t)(~rows &
28     ROW_MASK) << 16);
29
30     /* 3. Enable selected active column (RESET = LOW) */
31     HAL_GPIO_WritePin(GPIOA, COL_PINS[index],
32     GPIO_PIN_RESET);
33 }

```

Program 1.15: Matrix Column Multiplexing Logic

1.10 Exercise 10

The objective of Exercise 10 is to implement a continuous left-scrolling text animation of character "A" on the 8×8 LED matrix by integrating a second software timer.

1. Brief Description of the Solution

The dynamic scrolling effect is decoupled from display scanning through two independent software timer tasks:

1. **Task 1: Matrix Multiplexing** (Timer 0): Runs every 2 ms (setTimer0(2)), updating column activation sequentially to maintain a steady display refresh rate (62.5 Hz).
2. **Task 2: Buffer Shifting** (Timer 1): Configured with setTimer1(500), it expires every 500 ms (500 ticks×1 ms). When triggered, it invokes shift_matrix_left(), shifting all matrix buffer bytes leftward by one position with circular wrap-around.

2. Source Code Implementation

Matrix Buffer Left Shift Function:

```

1 void setTimer1(int duration) {
2     timer1_counter = duration / TIMER_CYCLE; // duration in
   ms
3     timer1_flag = 0;
4 }
5
6 void shift_matrix_left(void) {
7     uint8_t temp = matrix_buffer[0];
8     for (int i = 0; i < MAX_LED_MATRIX - 1; i++) {
9         matrix_buffer[i] = matrix_buffer[i + 1];
10    }
11    matrix_buffer[MAX_LED_MATRIX - 1] = temp;
12 }

```

Program 1.16: Function to shift the LED Matrix buffer array leftward

Main Application Loop Processing:

```

1 int main(void) {
2     /* Peripherals & Timer Initialization */
3     HAL_Init();
4     SystemClock_Config();
5     MX_GPIO_Init();
6     MX_TIM1_Init();
7
8     HAL_TIM_Base_Start_IT(&htim1); // Start TIM1 in
   Interrupt mode
9
10    setTimer0(2); // Scanning task: 2ms per column
11    setTimer1(500); // Animation task: 500ms shift interval
12
13    while (1) {
14        /* Task 1: Column Scanning Task (~62.5 Hz refresh)
   */
15        if (timer0_flag == 1) {
16            timer0_flag = 0;
17            setTimer0(2);
18            updateLEDMatrix(index_led_matrix);

```

```

19         index_led_matrix = (index_led_matrix + 1) %
MAX_LED_MATRIX;
20     }
21
22     /* Task 2: Left-Scrolling Animation Task (0.5s
interval) */
23     if (timer1_flag == 1) {
24         timer1_flag = 0;
25         setTimer1(500);
26         shift_matrix_left();
27     }
28 }
29 }

```

Program 1.17: Main Loop Execution with Dual Software Timers in main.c

Source Code Link:

- GitHub Repository: https://github.com/huyle01112006-wq/C03010-LE-ANH-HUY-24111/tree/LAB/lab_2_report